AI Invoice Assistant

Requirements: problem selection, user needs, scenarios, acceptance criteria

Version 0.5 - 3 October 2026

1. Problem selection

1.1 Problem statement

Company X needs to manage its invoices. To do so it has to manage **customers**, **products** and **invoices**, and it needs to know its **turnover** per customer, per product and per month. Doing this by hand in spreadsheets is slow and error-prone: wrong totals, duplicate numbers, and reports that must be added up manually.

1.2 Proposed solution

A **REST API** (FastAPI) through which a client application manages customers, products and invoices by sending natural-language messages to a local AI model (Ollama, *llama3.2:3b*, temperature 0). The AI asks for the data step by step; Python checks the result and returns a proposal with a summary. Nothing is saved by the chat: after the user confirms, the client sends the proposal to the plain create / update / delete endpoints, which check it again and save it. Turnover reports are computed by Python. The client can also send free questions about the invoices in plain language.

1.3 Why this problem

- Clear, everyday business case with structured data - well suited to test how far a small local LLM can go.
- Runs fully offline: no customer or financial data leaves the computer.
- Mix of conversation (AI) and strict rules (code) makes the division of responsibility visible.

1.4 Scope

In scope (v1)	Out of scope (v1)
Create, retrieve, update, delete customers	Generating invoice PDFs for sending to customers
Create, retrieve, update, delete products	VAT / taxes, discounts, multiple currencies
Create, retrieve, update, delete invoices with several lines	Partial payments, reminders, dunning
Turnover by customer and by product between two dates	Margin reports (StandardBuyPrice is stored only)
Turnover by year / month, optional filter by customer and product	Client application / user interface (built separately on top of the API)
Free questions about invoices in natural language	Multi-user, login, access from the network
REST API (FastAPI), reachable only from this computer	Database, cloud sync, accounting export
Local JSON files as storage	

1.5 Constraints and design decisions

ID	Decision	Reason
D1	Model: llama3.2:3b via Ollama, temperature 0	Given in the task; local and deterministic.
D2	One bot per area (customer, product, invoice, question) behind one REST API; reports are plain API calls without AI	Small models follow short, focused prompts much more reliably than one big prompt.
D3	Bots never talk to each other; Python is the hub, JSON files are the shared state	Simple and predictable; a 3B model cannot reliably coordinate with other agents.
D4	Numbers and codes (CUS-001, PRD-001, INV-001) are assigned by Python, never by the AI	Avoids invented or duplicate numbers. Numbers of deleted records are never reused.
D5	All arithmetic (line amounts, invoice totals, turnover, answers to questions) is done by Python	3B models are unreliable at calculation.
D6	The chat never saves. It returns a checked proposal with a summary; the client saves it with a separate create / update / delete call after the user confirms	In tests the bot skipped its own confirmation and deleted directly.
D7	Python validates every AI command before it is returned or saved	In tests the bot sent placeholders ("...") and accepted invalid values.
D8	Invoice lines store their own UnitPrice (default: product's StandardSellPrice, editable)	Later price changes must not alter existing invoices or past turnover.
D9	REST API built with FastAPI, reachable only from this computer (127.0.0.1) in v1	The final system must be usable by other applications; local only means no login is needed in v1.
D10	Two parts in one API: chat endpoints (AI, only propose) and CRUD endpoints (no AI, check again and save). Both use the same checks.	The data part works and can be tested without AI; other programs can use it directly.

2. User needs

User: an employee of company X who manages invoicing through a client application connected to the API, working alone on one computer.

ID	As a user, I want to...	so that...	Priority
N1	create, retrieve, update and delete customers	I don't retype their data on every invoice	Must
N2	create, retrieve, update and delete products with a standard sell and buy price	invoice lines are consistent and quick to add	Must
N3	create an invoice with several lines by answering simple questions one at a time	I don't have to know any format or form	Must
N4	have the unit price proposed from the product, but be able to change it on the invoice	I can give a special price without changing the product	Must
N5	have line amounts and invoice totals calculated automatically	I never send a wrong amount	Must
N6	retrieve, update and delete invoices (e.g. mark an invoice as paid)	my records stay up to date	Must
N7	see a summary and confirm before anything is saved or deleted	a misunderstanding by the AI can't damage my data	Must
N8	see the turnover per customer between two dates	I know which customers matter most	Must
N9	see the turnover per product between two dates	I know which products sell	Must
N10	see the turnover per month of a year, optionally for one customer and/or one product	I can follow the evolution over the year	Must
N11	ask free questions about invoices (e.g. how much a customer still owes, how many are unpaid)	I get quick answers without building a report	Should
N12	be told clearly when my input is invalid or something is not allowed	I know what to fix	Should
N13	keep my data locally and work offline	the company's data stays private	Must
N14	get answers quickly	using the assistant is faster than doing it by hand	Should

2.1 Data to manage

Entity	Fields	Notes
Customer	Number (CUS-001), Name, Address	Number generated by the system.
Product	Code (PRD-001), Name, StandardSellPrice, StandardBuyPrice	Code generated by the system. Prices in EUR, numbers >= 0. StandardBuyPrice is stored but not used in any report.
Invoice	Number (INV-001), Date, DueDate, Status (paid / unpaid), Customer	DueDate and Status are added to the given specification; they are needed for the turnover definition.
Invoice line	Line Number, Product, Quantity, UnitPrice	Line Numbers 1, 2, 3... per invoice. UnitPrice defaults to the product's StandardSellPrice and can be changed. Line amount = Quantity x UnitPrice.

2.2 Definition of turnover

Turnover = sum of (Quantity x UnitPrice) of the lines of **paid** invoices whose **DueDate** lies in the period (both boundary dates included). Unpaid invoices are not turnover. When filtered or grouped by **product**, only the lines of that product are counted. Dates are entered as DD/MM/YYYY.

2.3 Reports

ID	Report	Input	Output
R1	Turnover by customer	date1, date2	One row per customer: Customer | Turnover, then a Total row
R2	Turnover by product	date1, date2	One row per product: Product | Turnover, then a Total row
R3	Turnover by month	Year; optional Customer; optional Product (one, the other or both)	12 rows January ... December: Month | Turnover, then a Total row

3. Scenarios

Each scenario is a typical session. "Client" is the application the user works with; "API" means the Python program; "Bot" means the AI model.

S1	Add a new customer
Actor	User (through the client)
Precondition	The API is running.
Steps	1. The user sends "add a customer" to the customer chat (`POST /customers/chat`). 2. The bot asks for the name, then the address - one question per reply. 3. The API returns a proposal with the summary *Create customer: Helmut Schmidt | Dortmund, Hauptstr. 1*; the client shows it. Nothing is saved yet. 4. The user confirms; the client sends the proposal to `POST /customers`.
Expected result	Customer saved as CUS-00x in data/customers.json; the API returns the new number.

S2	Add a product
Actor	User (through the client)
Precondition	The API is running.
Steps	1. The user sends "new product" to the product chat (`POST /products/chat`). 2. The bot asks for name, standard sell price and standard buy price. 3. The user enters "abc" as sell price; the API rejects it and the bot asks again. 4. The API returns a proposal with the summary; the user confirms and the client sends it to `POST /products`.
Expected result	Product saved with a new code PRD-00x.

S3	Create an invoice with two lines and a special price
Actor	User (through the client)
Precondition	Customer Helmut and products "Laptop" (sell price 900) and "Mouse" (sell price 20) exist.
Steps	1. The user sends "invoice for Helmut" to the invoice chat (`POST /invoices/chat`). 2. The bot asks for the invoice date and the due date. 3. The bot asks for the first line: "2 Laptop"; unit price 900 is proposed; the user changes it to 850. 4. The bot asks for the next line: "5 Mouse", the user keeps 20. Then "no more lines". 5. The bot asks for the status (paid / unpaid). 6. The API returns a proposal with the lines (1: 2 x Laptop x 850 = 1700; 2: 5 x Mouse x 20 = 100) and the total 1800; the user confirms and the client sends it to `POST /invoices`.
Expected result	Invoice saved as INV-00x with line numbers 1 and 2 and total 1800. The product price of Laptop stays 900.

S4	Retrieve data
Actor	User (through the client)
Precondition	Customers, products and invoices exist.
Steps	1. The client calls `GET /customers`, `GET /products/PRD-002` and `GET /invoices/INV-001`.
Expected result	The API returns the records from the files; the invoice is returned with all lines and its total.

S5	Mark an invoice as paid
Actor	User (through the client)
Precondition	INV-002 exists and is unpaid.
Steps	1. The user sends "INV-002 was paid" to the invoice chat. 2. The API returns a proposal with the summary *Change status of INV-002: 'unpaid' -> 'paid'*. 3. The user confirms; the client sends `PUT /invoices/INV-002`.
Expected result	Status updated; INV-002 now counts in the turnover reports.

S6	Delete a customer who still has invoices
Actor	User (through the client)
Precondition	Wilfried (CUS-002) is used in INV-001.
Steps	1. The user sends "delete Wilfried" to the customer chat.
Expected result	The API refuses and explains that Wilfried is used in INV-001. Nothing is deleted.

S7	Turnover by customer (R1)
Actor	User (through the client)
Precondition	Paid and unpaid invoices exist in 2026.
Steps	1. The client calls `GET /reports/turnover-by-customer` with date1 = 01/01/2026 and date2 = 31/03/2026.
Expected result	Table Customer | Turnover with one row per customer and a Total row. Only paid invoices with a due date in the period are counted.

S8	Turnover by product (R2)
Actor	User (through the client)
Precondition	As S7.
Steps	1. The client calls `GET /reports/turnover-by-product` with the same two dates.
Expected result	Table Product | Turnover with a Total row. The Total equals the Total of R1 for the same period.

S9	Turnover by month with a filter (R3)
Actor	User (through the client)
Precondition	As S7.
Steps	1. The client calls `GET /reports/turnover-by-month` with year = 2026 and customer = Helmut (no product).
Expected result	Table January ... December with Helmut's paid turnover per month and a Total row.

S10	Ask a free question
Actor	User (through the client)
Precondition	Several invoices exist.
Steps	1. The user sends "How much does Helmut still owe?" and "How many invoices are unpaid?" (`POST /questions`).
Expected result	The bot answers using numbers pre-calculated by Python; the numbers match the data exactly.

S11	The bot makes a mistake
Actor	API
Precondition	Any conversation.
Steps	1. The bot sends an incomplete command (e.g. a value "...") or an unknown number. 2. The API rejects it, tells the bot what is wrong, and the bot asks the user again.
Expected result	Nothing is saved; the client only receives a normal follow-up question.

S12	User changes their mind
Actor	User (through the client)
Precondition	A proposal with its summary was returned.
Steps	1. The user answers no; the client does not send the proposal.
Expected result	Nothing is changed.

4. Acceptance criteria

Each criterion is testable. The software is accepted when all criteria pass.

4.1 General

ID	Criterion	Needs
AC-G1	The chat endpoints never create, change or delete anything; they only return a proposal with a summary. Records change only through the create / update / delete endpoints.	N7
AC-G2	Every number / code is generated by the system, unique and never reused after deletion.	N1-N6
AC-G3	An invalid or incomplete AI command is never saved; the user is asked again instead.	N12
AC-G4	The application works with the network disconnected; data is stored only in data/*.json.	N13
AC-G5	Internal data blocks (e.g. <CUSTOMER DATA>) are never returned by the API.	N12
AC-G6	Invalid input returns a clear error message (HTTP 4xx); the API never crashes on unexpected input.	N12
AC-G7	A bot reply arrives within 10 seconds on this computer.	N14
AC-G8	The API is reachable only from this computer (127.0.0.1).	N13
AC-G9	The create / update / delete endpoints run the same checks as the chat; invalid data sent to them directly is rejected with a clear message (HTTP 4xx).	N12

4.2 Customers

ID	Criterion	Needs
AC-C1	Given no customers, when the user completes S1, then data/customers.json contains one customer CUS-001 with the entered Name and Address.	N1
AC-C2	Name and Address are required; an empty value is never saved.	N1, N12
AC-C3	The Number cannot be changed.	N1
AC-C4	A customer used in any invoice cannot be deleted; the message names the invoice(s).	N1, N12
AC-C5	Referring to a non-existing customer results in a clear message, not an invented customer.	N12
AC-C6	The API returns the list of all customers and a single customer.	N1

4.3 Products

ID	Criterion	Needs
AC-P1	A product is saved with Name, StandardSellPrice and StandardBuyPrice; prices must be numbers >= 0, anything else is rejected.	N2
AC-P2	The Code cannot be changed.	N2
AC-P3	A product used in any invoice cannot be deleted.	N2, N12
AC-P4	Changing a product's StandardSellPrice does not change the UnitPrice of existing invoice lines.	N4
AC-P5	The API returns the list of all products and a single product.	N2

4.4 Invoices

ID	Criterion	Needs
AC-I1	An invoice can only reference an existing customer and existing products.	N3
AC-I2	An invoice has at least one line; every Quantity is a whole number > 0.	N3
AC-I3	Lines are numbered 1, 2, 3... without gaps (also after a line is removed).	N3
AC-I4	A new line's UnitPrice is the product's StandardSellPrice unless the user enters another price >= 0.	N4
AC-I5	Each line amount equals Quantity x UnitPrice and the invoice total equals the sum of the line amounts, exact to the cent.	N5
AC-I6	Date and DueDate are valid dates; DueDate is not before Date.	N3, N12
AC-I7	Status is only "paid" or "unpaid" and can be changed after creation.	N6
AC-I8	The API returns the list of all invoices and a single invoice with all its lines and total.	N6

4.5 Turnover reports

ID	Criterion	Needs
AC-R1	Only lines of paid invoices whose DueDate is between date1 and date2 (both included) are counted.	N8-N10
AC-R2	R1 shows one row per customer and a Total row; R2 one row per product and a Total row. Each Total equals the sum of its rows.	N8, N9
AC-R3	For the same period, the Total of R1 equals the Total of R2.	N8, N9
AC-R4	R3 shows the 12 months January to December and a Total row; without filters, the R3 Total for a year equals the R1 Total for 01/01 to 31/12 of that year.	N10
AC-R5	With a customer filter, R3 counts only that customer's invoices; with a product filter, only the lines of that product; both filters can be combined.	N10
AC-R6	Dates are entered as DD/MM/YYYY; an invalid date or date1 after date2 is rejected with a clear message.	N8, N9, N12

4.6 Free questions

ID	Criterion	Needs
AC-Q1	"How many invoices are unpaid?" returns the exact count.	N11
AC-Q2	"How much does <customer> owe?" returns the exact sum of that customer's unpaid invoices.	N11
AC-Q3	"What is the sum of all invoices?" returns exactly the sum computed from data/invoices.json.	N11
AC-Q4	For a question the data cannot answer, the bot says it doesn't know instead of inventing a number.	N11